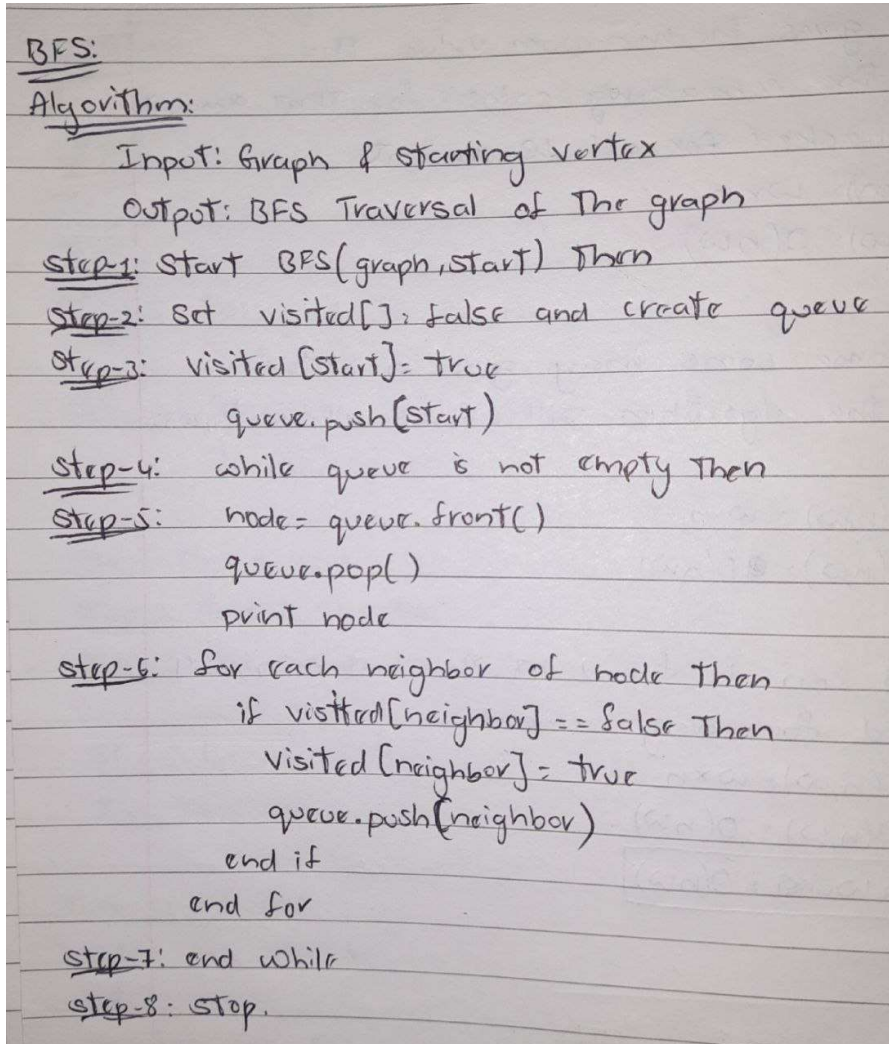


PRACTICAL-08

AIM: Implementation of Graph and Searching (DFS and BFS).

1.) BFS:

ALGORITHM:



BFS:
Algorithm:
Input: Graph & starting vertex
Output: BFS Traversal of The graph
step-1: Start BFS(graph, start) Then
step-2: Set visited[]: false and create queue
step-3: visited[start] = true
 queue.push(start)
step-4: while queue is not empty Then
step-5: node = queue.front()
 queue.pop()
 print node
step-6: for each neighbor of node Then
 if visited[neighbor] == false Then
 visited[neighbor] = true
 queue.push(neighbor)
 end if
 end for
step-7: end while
step-8: stop.

PROGRAM:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <queue>

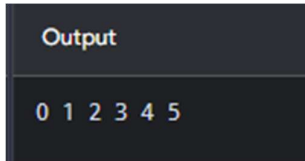
using namespace std;

void BFS(vector<vector<int>> graph, int start)
{
    bool visited[6] = {false};
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty())
    {
        int node = q.front();
        q.pop();
        cout << node << " ";
        for (int x : graph[node])
        {
            if (!visited[x])
            {
                visited[x] = true;
                q.push(x);
            }
        }
    }
}

int main()
{
    vector<vector<int>> graph = {
        {1, 2},
        {0, 3, 4},
        {0, 5},
```

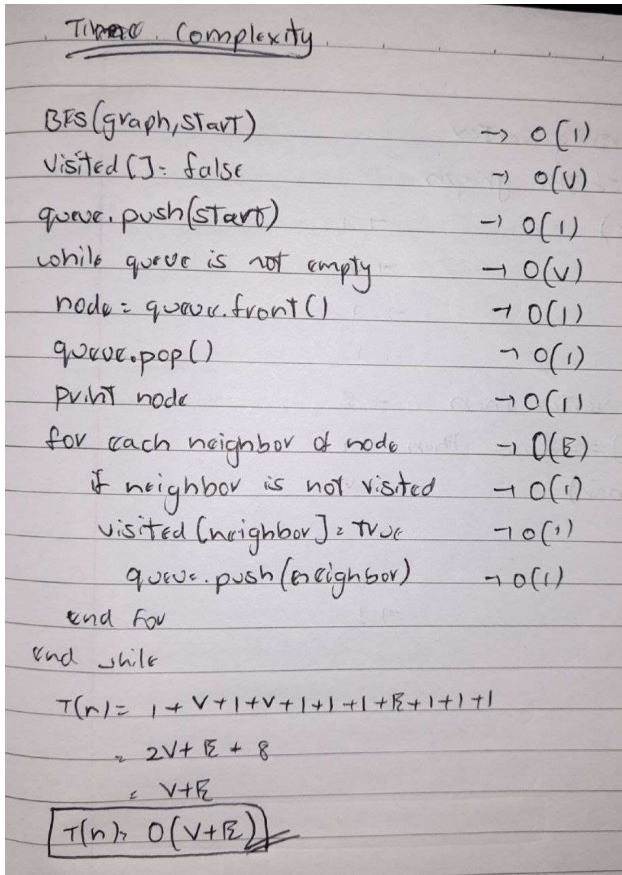
```
{1},  
{1},  
{2}  
};  
BFS(graph, 0);  
return 0;  
}
```

OUTPUT:



Output
0 1 2 3 4 5

TIME COMPLEXITY:



Time Complexity

BFS(graph, start)	→ $O(1)$
visited[] = false	→ $O(V)$
queue.push(start)	→ $O(1)$
while queue is not empty	→ $O(V)$
node = queue.front()	→ $O(1)$
queue.pop()	→ $O(1)$
print node	→ $O(1)$
for each neighbor of node	→ $O(E)$
if neighbor is not visited	→ $O(1)$
visited[neighbor] = true	→ $O(1)$
queue.push(neighbor)	→ $O(1)$
end for	
end while	

$$T(n) = 1 + V + 1 + V + 1 + 1 + 1 + E + 1 + 1 + 1$$
$$= 2V + E + 8$$
$$= V + E$$
$$T(n) = O(V + E)$$

Best case: Only The starting vertex is processed
 $T(n) = 1 + 1 + 1 = 3$
 $T(n) = O(1)$

Average case: Some or all vertices and edges are processed
 $T(n) = V + E + V$
 $= 2V + E$
 $T(n) = O(V + E)$

worst case: All vertices and all edges are processed
 $T(n) = V + E + V$
 $= 2V + E$
 $T(n) = O(V + E)$

2.) DFS

ALGORITHM:

DFS
Algorithm:
 Input: graph and starting vertex
 Output: DFS traversal of the graph

Step-1: Start DFS(graph, vertex) $\rightarrow 1$

Step-2: Set visited[] = false $\rightarrow V$

Step-3: visited[vertex] = true $\rightarrow 1$

Step-4: print vertex $\rightarrow 1$

Step-5: For each neighbor of vertex then $\rightarrow E$
 If visited[neighbor] == false then $\rightarrow 1$
 DFS(graph, neighbor) $\rightarrow V$
 end if
 end for

Step-6: stop. $\rightarrow 1$

PROGRAM:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;

void DFS(vector<vector<int>> graph, int vertex, bool visited[])
{
    visited[vertex] = true;
    cout << vertex << " ";
    for (int neighbor : graph[vertex])
    {
        if (!visited[neighbor])
        {
            DFS(graph, neighbor, visited);
        }
    }
}

int main()
{
    vector<vector<int>> graph = {

        {1, 2},
        {0, 3, 4},
        {0, 5},
        {1},
        {1},
        {2}
    };
    bool visited[6] = {false};
    DFS(graph, 0, visited);
    return 0;
}
```

OUTPUT:

```
Output
0 1 3 4 2 5
```

TIME COMPLEXITY:

Time complexity:

$$T(n) = 1 + V + 1 + 1 + E + 1 + V + 1$$
$$= 2V + E + 5$$
$$= V + E$$
$$T(n) = O(V + E)$$

Best case:

If the starting vertex has no edges

$$T(n) = 1 \quad \text{-- visiting starting vertex.}$$
$$T(n) = O(1)$$

Average case: DFS visits the required vertices and checks their edges:

$$T(n) = V + E$$
$$T(n) = O(V + E)$$

Worst case: All vertices and all edges are visited:

$$T(n) = V + E$$
$$T(n) = O(V + E)$$

CONCLUSION:

BFS and DFS are graph traversal techniques used to visit all the vertices of a graph. BFS uses a queue to visit vertices level by level, while DFS uses recursion/stack to explore deeply. Both are useful for solving different graph problems and have $O(V + E)$ time complexity using an adjacency list.